

Courte présentation

Le langage Go a été créé par Google en 2009 (17 ans)

Différencier les langages compilés et interprétés.

Langages compilés:

- Exemples: C, C++, Go
- Demande une compilation à chaque modification
- Plus rigide
- Plus complexes

Langages interprétés:

- Exemples: Python
- Risques de perte de mémoire

Gestion de mémoire

Java utilise un garbage collector, qui va permettre de se débarrasser de la mémoire qui n'est plus utilisée. Java utilise aussi une JVM, pour Java Virtual Machine.

Objectifs du Go

Un langage qui fonctionne :

- à bas niveau
- en web
- sur mobile
- avec de bonnes performances

Composition et héritage

Dans Go, on va préférer utiliser la composition, par rapport à l'héritage de la POO.

Principes de Go

- Simplicité
 - à écrire
 - à lire
- Rapidité

- d'exécution
- de complétion
- Rigidité
 - dans le formatage
 - dans la sécurité
- Pragmatisme
 - Une seule façon de faire
 - Avantages: stabilité et compatibilité

La concurrence

- Définition : plusieurs threads (processus) qui accèdent à la même ressource mémoire
- Pose des problèmes en termes de RAM;
 - Données corrompues
 - Données bloquées
- Solution : processus tâches, multi-threading

Go routines

Plusieurs tâches multithreading

Outils utilisant ou développés en Go

- Terraform
- Kubernetes
- Prometheus
- Daemons systèmes
- Agents DevOps

Code

Voilà un exemple de code, affichant un `Hello, world!`, 64 puissance 8, la date et l'heure et enfin l'âge. Étant donné que l'on n'avait pas encore vu les Strings, j'ai juste `Println` le nombre d'années, de mois et de jours qui séparent aujourd'hui de mon anniversaire.

```
package main
import (
    "fmt"
    "math"
    "time"
)
```

```
func main() {
    // Hello, world !
    fmt.Println("Hello, world !")

    // 64 puissance 8
    fmt.Println(math.Pow(64, 8))

    current_time := time.Now().Local()
    // Afficher la date et l'heure
    fmt.Println(current_time.Date())
    // Age
    birth_date := time.Date(2003, time.April, 13, 12, 35, 0, 0, time.UTC)
    age_year := current_time.Year() - birth_date.Year()
    age_month := current_time.Month() - birth_date.Month()
    age_day := current_time.Day() - birth_date.Day()
    fmt.Println(age_year, int(age_month), age_day)
}
```

La commande `go run` permet de compiler et de lancer un programme écrit en go.

```
rourou@LAPTOP-U6739NS7:~/go$ go run ./main.go
Hello, world !
2.81474976710656e+14
2026 May 26
23 1 13
```

La commande `go build -ldflags "-s -w" main.go` permet de compiler une version directement exécutable par la machine :

```
rourou@LAPTOP-U6739NS7:~/go$ go build -ldflags "-s -w" main.go
rourou@LAPTOP-U6739NS7:~/go$ ./main
Hello, world !
2.81474976710656e+14
2026 May 26
23 1 13
```

Les options permettent de supprimer la table des symboles (`-s`) et les infos de debug (`-w`) pour produire un binaire plus petit. Cependant, le débogage est ensuite impossible.

Types en Go

Entiers

Ces entiers sont signés:

- `int8`
- `int16`
- `int32`

Il existe aussi la possibilité d'avoir un non signé; on va simplement rajouter `u` devant le

nom du type:

- uint8
- uint16
- uint32
- uint64
- uint128

Enfin, il existe les types `uint` et `int`, dont le nombre de bits sera basé sur celui de la machine.

Floats

- float32
- float64

Booléens

`true` ou `false`, instanciable avec `var bool1 = true`

String

Instanciable avec `var texte = "Cours ESGI"`.

Possibilité de concaténer et d'additionner les `Strings`

Les `Strings` sont indexables

Spéciaux

Alias

- `byte = uint8`
- `rune = int32`

Type vide

Pour `String` : `""`

Pour `Boolean` : `false`

On utilise `Nil`

Déclarer une variable

La déclaration d'une variable se fait principalement avec le mot-clé `var`

L'initialisation se fait grâce à `:=` qui permet aussi de déclarer.

On peut aussi déclarer une constante avec le mot-clé `const`

Constante type "iota"

`iota` est un compteur automatique utilisé dans les constantes : il vaut 0 sur la première ligne, puis s'incrémente de 1 à chaque ligne suivante. Cela permet créer des énumérations sans préciser les valeurs à la main. La documentation de Go l'explique ici avec un super gif :

<https://go.dev/wiki/Iota>